ALL COMMANDS DevOps job ki daily-use + real-time troubleshooting

PART 1 — DOCKER
Step 1 — Basic Commands
docker --version
Docker install ayyinda, current Docker version enti ani check cheyyadaniki.
docker info
Docker daemon status, containers, images, storage, runtime and system-level Docker information choodadaniki.
docker ps
Currently running containers anni choodadaniki.
docker ps -a
Running + stopped/exited containers anni choodadaniki.
docker images
Local machine lo unna Docker images anni choodadaniki.
docker pull nginx:latest
Docker registry nunchi nginx:latest image local machine ki download cheyyadaniki.
docker image ls
Local images list choodadaniki. docker images ki alternative.
docker build -t mycompany/netflix:v1 .
Current directory Dockerfile ni use chesi mycompany/netflix:v1 Docker image build cheyyadaniki.
docker run -d --name netflix -p 8080:80 mycompany/netflix:v1
Image nunchi container create chesi background lo run chestundi. Host port 8080 ni container port 80 ki map chestundi.
docker start netflix
Already stopped unna netflix container ni start cheyyadaniki.
docker stop netflix
Running netflix container ni gracefully stop cheyyadaniki.
docker restart netflix
Container ni stop chesi immediate ga malli start cheyyadaniki.
docker rm netflix
Stopped container ni remove cheyyadaniki.
docker rm -f netflix
Running container unna kuda force ga remove cheyyadaniki.
docker rmi mycompany/netflix:v1
Local Docker image remove cheyyadaniki.
docker logs netflix
Container application stdout/stderr logs choodadaniki.
docker logs -f netflix
Container logs live ga follow cheyyadaniki.
docker exec -it netflix /bin/bash
Running container lopalaki interactive shell tho enter avvadaniki.
docker exec -it netflix /bin/sh
bash available lenappudu lightweight sh shell tho container lopalaki velladaniki.
docker inspect netflix
Container configuration, network, mounts, IP, environment and runtime details JSON format lo choodadaniki.
docker stats
Running containers CPU, memory, network and disk I/O usage live ga choodadaniki.
docker top netflix
Container lopala currently running processes choodadaniki.
docker port netflix
Container ports host machine ki ela mapped unnayo choodadaniki.
docker rename netflix netflix-prod
Existing container name change cheyyadaniki.
docker cp netflix:/usr/share/nginx/html/index.html .
Container lopala file ni host machine ki copy cheyyadaniki.
docker cp ./index.html netflix:/usr/share/nginx/html/
Host machine nunchi file ni container lopalaki copy cheyyadaniki.
docker login
Docker registry ki authenticate avvadaniki.
docker tag netflix:v1 mycompany/netflix:v1
Existing image ki repository/tag name assign cheyyadaniki.
docker push mycompany/netflix:v1
Local image ni registry ki upload cheyyadaniki.
docker history mycompany/netflix:v1
Image build lo create ayina layers/history choodadaniki.
docker system df
Docker images, containers, volumes and build cache entha disk space use chestunnayo choodadaniki.
docker system prune
Unused Docker resources cleanup cheyyadaniki. Production lo blindly use cheyyakudadhu.

Step 2 — Docker Real-time Commands
New application image build cheyyali

docker build -t mycompany/netflix:v2.3.0 .
Developer new application code ichinappudu Dockerfile base chesi new production image create chestam.
docker run -d --name netflix -p 8080:80 mycompany/netflix:v2.3.0
Build chesina new image nunchi container start chesi application local ga verify chestam.
Existing application new version ki update cheyyali
docker stop netflix
Old container stop chestundi.
docker rm netflix
Old container remove chestundi.
docker run -d --name netflix -p 8080:80 mycompany/netflix:v2.3.0
New image tho new container create chestundi.
Production Kubernetes environment lo usually direct docker stop/rm/run cheyyamu; Kubernetes deployment update chestundi.
Image registry ki release cheyyali
docker build -t mycompany/netflix:v2.3.0 .
New release image build chestundi.
docker login
Registry authentication chestundi.
docker push mycompany/netflix:v2.3.0
Image registry ki upload chestundi.
Kubernetes later mycompany/netflix:v2.3.0 image ni registry nunchi pull chesi Pods lo containers start chestundi.
Container health check cheyyali
docker ps
Container running lo unda check chestam.
docker logs netflix
Application startup/error logs check chestam.
docker inspect netflix
Configuration, network and state details check chestam.
docker stats netflix
CPU/memory abnormal ga use chestunda check chestam.

Step 3 — Docker Troubleshooting
Problem: Container immediately exit avtundi
docker ps -a
Container Exited (1), Exited (137) etc. ani status lo choodachu.
docker logs netflix
Actual application error usually ikkada dorukutundi.
docker inspect netflix
Exit code and container configuration verify cheyyadaniki.
docker start netflix
Container restart cheyyadaniki, but underlying error fix kakapothe malli exit avvachu.
Example: npm start command fail ayithe logs lo application startup error kanipistundi. App code/configuration fix chesi image rebuild cheyyali.
Problem: Website open avvatledu
docker ps
Container actually running lo unda check cheyyali.
docker port netflix
Port mapping correct aa check cheyyali.
curl http://localhost:8080
Host nunchi application response vastunda test cheyyali.
docker logs netflix
Nginx/application errors check cheyyali.
Problem: Port already allocated
docker run -d -p 8080:80 --name netflix mycompany/netflix:v1
port is already allocated error vaste host port 8080 already another process/container use chestundi.
docker ps
Existing container port mapping choodali.
docker ps --format "table {{.Names}}\t{{.Ports}}"
Which container which host port use chestundo clear ga choodali.
docker run -d -p 8081:80 --name netflix mycompany/netflix:v1
Alternative free host port use cheyyachu.
Problem: ImagePull / image not found
docker pull mycompany/netflix:v2.3.0
Registry lo exact image/tag available aa test chestundi.
docker images
Local image already unda check chestundi.
docker login
Private registry authentication verify chestundi.
Usually issue **wrong repository name, wrong tag, authentication, or registry URL**.

PART 2 — GIT
Step 1 — Basic Commands

git --version
Git installed/version check.
git config --global user.name "Your Name"
Git commits lo author name configure chestundi.
git config --global user.email "you@example.com"
Git commit author email configure chestundi.
git clone <repository-url>
Remote repository complete copy local machine ki download chestundi.
git status
Working directory lo modified, untracked, staged files enti ani check chestundi.
git branch
Local branches list chestundi.
git branch -a
Local + remote branches list chestundi.
git switch main
main branch ki switch chestundi.
git switch -c feature/netflix-login
New branch create chesi immediately aa branch ki switch chestundi.
git pull
Remote repository latest changes ni current branch ki fetch + integrate chestundi.
git fetch
Remote changes download chestundi but working branch ni automatically change cheyyadu.
git add .
Modified/new files ni staging area ki add chestundi.
git commit -m "Update Netflix login"
Staged changes ni Git history lo commit chestundi.
git push origin feature/netflix-login
Local branch changes remote repository ki upload chestundi.
git log --oneline
Commit history short format lo choodadaniki.
git diff
Working directory lo unstaged changes choodadaniki.
git diff --staged
Staged changes choodadaniki.
git restore file.txt
Uncommitted file changes discard cheyyadaniki.
git stash
Temporary ga uncommitted changes side lo save chesi working tree clean cheyyadaniki.
git stash pop
Previously stashed changes malli apply cheyyadaniki.

Step 2 — Git Real-time
Developer feature complete chesadu
git status
Em changes unnayo check chestam.
git switch -c feature/payment
Feature kosam separate branch create chestam.
git add .
Changes stage chestam.
git commit -m "Add payment integration"
Logical change commit chestam.
git push -u origin feature/payment
Remote branch create/push chestam. Tarvata GitHub/GitLab lo Pull Request create chestaru.
Latest production code teesukovali
git switch main
Main branch ki vellali.
git pull origin main
Latest main code local ki teesukovali.
Commit history investigate cheyyali
git log --oneline --decorate --graph --all
Branches and commit history visual-style text format lo investigate cheyyadaniki.

Step 3 — Git Troubleshooting
Merge conflict
git status
Conflict unna files identify cheyyadaniki.
Conflict file open chesthe:
<<<<<<< HEAD
your changes
=======
incoming changes

>>>>>>> main
Correct code manually select chesi conflict markers remove cheyyali.
git add <file>
Resolved file stage chestundi.
git commit
Merge resolution complete chestundi.
Wrong commit local lo chesav
git log --oneline
Wrong commit identify cheyyadaniki.
git revert <commit-id>
Already shared/pushed commit ni safely reverse cheyyadaniki preferred method.
Push rejected
git pull --rebase origin main
Remote changes first integrate chesi local commits ni replay chestundi.
git push origin main
Updated history push chestundi.

PART 3 — LINUX
Step 1 — Basic Commands
pwd
Current directory path choodadaniki.
ls
Current directory files/folders choodadaniki.
ls -la
Hidden files tho complete listing choodadaniki.
cd /var/log
Directory change cheyyadaniki.
mkdir app
New directory create cheyyadaniki.
touch app.log
Empty file create cheyyadaniki.
cp app.log backup.log
File copy cheyyadaniki.
mv app.log /tmp/
File move/rename cheyyadaniki.
rm app.log
File delete cheyyadaniki.
rm -rf old-directory
Directory and contents recursively remove cheyyadaniki. **Very dangerous command.**
cat app.log
File content print cheyyadaniki.
less app.log
Large file paginated ga choodadaniki.
head app.log
File first lines choodadaniki.
tail app.log
Last lines choodadaniki.
tail -f app.log
Log file new entries live ga follow cheyyadaniki.
grep "ERROR" app.log
File lo ERROR unna lines search cheyyadaniki.
find /var/log -name "*.log"
Log files locate cheyyadaniki.
chmod +x script.sh
Script executable permission ivvadaniki.
chown user:user app.log
File owner/group change cheyyadaniki.
ps aux
Running processes anni choodadaniki.
top
CPU/memory usage and processes live ga monitor cheyyadaniki.
df -h
Filesystem disk space usage choodadaniki.
du -sh /var/log
Specific directory entha disk space use chestundo choodadaniki.
free -h
RAM and swap usage choodadaniki.
uptime
System uptime and load average choodadaniki.

Step 2 — Linux Real-time

Server lo application process check
ps aux | grep nginx
Nginx processes running lo unnaya check cheyyadaniki.
systemctl status nginx
Nginx service running/failed state choodadaniki.
systemctl restart nginx
Configuration/code change tarvata Nginx restart cheyyadaniki.
journalctl -u nginx
Nginx systemd logs choodadaniki.
Server disk full issue
df -h
Which filesystem full ayyindo identify chestam.
du -sh /* 2>/dev/null
Top-level directories lo ekkuva space ekkada use avtundo identify chestam.
du -sh /var/log/*
Logs excessive disk space consume chestunnaya check chestam.

Step 3 — Linux Troubleshooting
CPU high
top
CPU ekkuva use chestunna process identify chestam.
ps aux --sort=-%cpu | head
Highest CPU processes top lo chupistundi.
Memory high
free -h
RAM/swap available state check chestam.
ps aux --sort=-%mem | head
Highest memory consuming processes identify chestam.
Disk full
df -h
Filesystem identify chestam.
du -xh /var | sort -h | tail
/var lo large directories/files identify chestam.

PART 4 — KUBERNETES
Step 1 — Basic Commands
kubectl version
kubectl/client and cluster version information check cheyyadaniki.
kubectl cluster-info
Kubernetes cluster reachable aa check cheyyadaniki.
kubectl get nodes
Cluster worker/control-plane nodes status choodadaniki.
kubectl get namespaces
Cluster namespaces list choodadaniki.
kubectl get pods
Current namespace lo Pods choodadaniki.
kubectl get pods -A
All namespaces lo Pods choodadaniki.
kubectl get pods -o wide
Pods tho paatu IP, node etc. information choodadaniki.
kubectl get deployments
Deployments list/status choodadaniki.
kubectl get services
Services list choodadaniki.
kubectl get ingress
Ingress resources choodadaniki.
kubectl get configmaps
ConfigMaps list choodadaniki.
kubectl get secrets
Secrets list choodadaniki.
kubectl describe pod <pod-name>
Specific Pod events, image, container state, probes, scheduling information detailed ga choodadaniki.
kubectl logs <pod-name>
Pod container application logs choodadaniki.
kubectl logs -f <pod-name>
Logs live ga follow cheyyadaniki.
kubectl exec -it <pod-name> -- /bin/sh
Running Pod container lopalaki shell tho enter avvadaniki.
kubectl apply -f deployment.yaml
YAML manifest create/update cheyyadaniki.
kubectl delete -f deployment.yaml

Manifest lo defined resource remove cheyyadaniki.
kubectl get rs
ReplicaSets choodadaniki.
kubectl get events --sort-by=.lastTimestamp
Recent Kubernetes events chronological ga investigate cheyyadaniki.
kubectl get all
Current namespace lo common workload/service resources summary choodadaniki.

Step 2 — Kubernetes Real-time
3 Pods → 6 Pods
kubectl scale deployment netflix --replicas=6
Existing netflix Deployment desired replicas 3 nunchi 6 ki immediate ga change chestundi.
kubectl get deployment netflix
Deployment desired/current/ready replicas verify cheyyadaniki.
kubectl get pods -l app=netflix
Netflix label unna Pods anni choodadaniki.
kubectl get pods -l app=netflix -w
Pod creation live ga watch cheyyadaniki.
YAML permanent ga 6 replicas maintain cheyyali
deployment.yaml lo:
```
spec:
  replicas: 6
```
Tarvata:
kubectl apply -f deployment.yaml
Deployment configuration update chestundi.
kubectl rollout status deployment/netflix
Deployment rollout complete ayyinda check chestundi.

New image version deploy cheyyali
kubectl set image deployment/netflix netflix=mycompany/netflix:v2.3.0
Existing Deployment lo netflix container image ni v2.3.0 ki update chestundi.
kubectl rollout status deployment/netflix
New Pods successful ga rollout avtunnaya check chestundi.
kubectl get pods
New Pods running/ready state verify chestam.

Previous version ki rollback
kubectl rollout history deployment/netflix
Deployment rollout revisions choodadaniki.
kubectl rollout undo deployment/netflix
Previous working revision ki rollback cheyyadaniki.
kubectl rollout status deployment/netflix
Rollback complete ayyinda verify cheyyadaniki.

Application logs
kubectl logs deployment/netflix
Deployment associated Pod logs inspect cheyyadaniki.
kubectl logs <pod-name> --previous
Container restart ayyaka previous crashed container logs choodadaniki.

Step 3 — Kubernetes Troubleshooting
Pod CrashLoopBackOff
First:
kubectl get pods
Pod current status choodali.
Example:
```
netflix-abc123  0/1  CrashLoopBackOff
```
Next:
kubectl logs netflix-abc123
Application crash reason identify cheyyali.
If container already restarted:
kubectl logs netflix-abc123 --previous
Previous crashed instance logs choodali.
Then:
kubectl describe pod netflix-abc123
Events, probes, image, mounts, environment, scheduling details check cheyyali.
Possible causes:

- application startup error
- wrong environment variable
- Secret/ConfigMap problem

- wrong command
- missing file
- database connection failure
- failed health probe

Pod Pending
kubectl get pods
Pending status identify cheyyali.
kubectl describe pod <pod-name>
Bottom lo **Events** section check cheyyali.
Typical issue:

- insufficient CPU/memory
- node selector mismatch
- taint/toleration problem
- PVC not available

Nodes resources:
kubectl get nodes
Node availability check.
kubectl describe node <node-name>
Node capacity, allocatable resources, taints etc. check.

ImagePullBackOff
kubectl describe pod <pod-name>
Events lo exact image pull error choodali.
For example:
Failed to pull image
Then Deployment image verify:
kubectl get deployment netflix -o yaml
Image repository/tag correct aa check cheyyali.
Registry private ayithe image pull secret configuration check cheyyali.

Service request reach avvatledu
kubectl get svc
Service exists aa check.
kubectl describe svc netflix-service
Service selector and endpoints inspect cheyyali.
kubectl get endpoints netflix-service
Service ki backend Pods attach ayyaya check cheyyali.
If endpoints empty:
kubectl get pods --show-labels
Service selector and Pod labels match avtunnaya check cheyyali.

Ingress 502/503
kubectl get ingress
Ingress exists aa check.
kubectl describe ingress netflix-ingress
Host/path/backend configuration and events check cheyyali.
kubectl get svc netflix-service
Correct Service unda check.
kubectl get endpoints netflix-service
Service backend Pods ki point chestunda check.
kubectl get pods
Backend Pods Ready aa check.
Flow:
Ingress
↓
Service
↓
Endpoints
↓
Pods
↓
Container
Ee chain lo ekkada break ayyindo identify chestam.

PART 5 — HELM
Step 1 — Basic Commands
helm version
Helm installed/version check.

helm repo list
Configured Helm repositories list.
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
Helm chart repository add cheyyadaniki.
helm repo update
Configured repositories latest chart metadata download chestundi.
helm search repo prometheus
Available charts search chestundi.
helm list
Current namespace lo installed releases choodadaniki.
helm list -A
All namespaces lo Helm releases choodadaniki.
helm show values <chart>
Chart default configurable values choodadaniki.
helm install netflix ./netflix
Local Helm chart ni netflix release name tho install chestundi.
helm upgrade netflix ./netflix
Existing release ni new chart/configuration tho update chestundi.
helm uninstall netflix
Helm release and associated resources remove cheyyadaniki.

Step 2 — Helm Real-time
Production replicas change
helm upgrade netflix ./netflix --set replicaCount=6
Existing Helm release ni 6 replicas configuration tho upgrade chestundi.
Production image version change
helm upgrade netflix ./netflix --set image.tag=v2.3.0
Application image tag v2.3.0 ki change chestundi.
Values file use
helm upgrade netflix ./netflix -f values-prod.yaml
Production-specific values file use chesi release update chestundi.

Step 3 — Helm Troubleshooting
helm status netflix
Release deployed/failed state and basic information choodadaniki.
helm history netflix
Previous Helm revisions choodadaniki.
helm rollback netflix 3
Release ni revision 3 ki rollback cheyyadaniki.
helm get values netflix
Current release lo applied values choodadaniki.
helm get manifest netflix
Helm generate chesina actual Kubernetes manifests choodadaniki.

PART 6 — TERRAFORM
Step 1 — Basic Commands
terraform version
Terraform version check.
terraform init
Provider modules/backend dependencies initialize/download chestundi.
terraform fmt
Terraform files formatting standardize chestundi.
terraform validate
Terraform configuration syntax and internal consistency validate chestundi.
terraform plan
Infrastructure lo Terraform em create/change/delete cheyyabotundo preview chestundi.
terraform apply
Planned infrastructure changes actual ga apply chestundi.
terraform destroy
Terraform manage chestunna infrastructure resources destroy cheyyadaniki.
terraform show
Current Terraform state details choodadaniki.
terraform state list
Terraform state lo tracked resources list chestundi.
terraform output
Terraform outputs choodadaniki.

Step 2 — Terraform Real-time
New infrastructure change
terraform init
Project dependencies/provider initialize.

terraform fmt
Code formatting.
terraform validate
Configuration valid aa check.
terraform plan
Actual changes apply cheyyakunda preview.
terraform apply
Approved infrastructure changes create/update.
Specific resource inspect
terraform state list
State lo resource address identify cheyyadaniki.
terraform state show aws_instance.web
Specific resource state details inspect cheyyadaniki.

Step 3 — Terraform Troubleshooting
Plan unexpected change chupistundi
terraform plan
Terraform desired configuration and actual state difference identify cheyyadaniki.
terraform state list
Resource Terraform state lo tracked aa check.
terraform state show <resource>
State lo stored attributes inspect cheyyadaniki.
State lock problem
terraform force-unlock <LOCK_ID>
Stale state lock remove cheyyadaniki.
Production lo lock actually active unda verify chesi matrame use cheyyali.

PART 7 — JENKINS
Step 1 — Basic Commands
Jenkins mostly web UI/pipeline based tool, so Linux/agent side commands kuda important.
systemctl status jenkins
Jenkins service running aa check cheyyadaniki.
systemctl restart jenkins
Jenkins restart cheyyadaniki.
journalctl -u jenkins
Jenkins service logs choodadaniki.
Pipeline lo commonly:
git clone <repository-url>
Source code Jenkins workspace ki download cheyyadaniki.
docker build -t mycompany/netflix:${BUILD_NUMBER} .
CI pipeline lo application Docker image build cheyyadaniki.
docker push mycompany/netflix:${BUILD_NUMBER}
Built image registry ki push cheyyadaniki.
kubectl set image deployment/netflix netflix=mycompany/netflix:${BUILD_NUMBER}
CI/CD pipeline nunchi Kubernetes Deployment image update cheyyadaniki.

Step 2 — Jenkins Real-time CI/CD Flow
Typical pipeline:
Developer
↓
Git Push
↓
Jenkins
↓
Checkout
↓
Build
↓
Test
↓
Docker Build
↓
Docker Push
↓
Kubernetes Deploy
↓
Rollout Verification
Pipeline commands conceptually:
git checkout main
Correct branch code checkout.
npm ci

package-lock.json based on clean dependency installation.
npm run build
Application production build.
docker build -t mycompany/netflix:$BUILD_NUMBER .
Unique CI build image create.
docker push mycompany/netflix:$BUILD_NUMBER
Image registry ki upload.
kubectl set image deployment/netflix netflix=mycompany/netflix:$BUILD_NUMBER
Kubernetes new image deploy.
kubectl rollout status deployment/netflix
Deployment successful aa verify.

Step 3 — Jenkins Troubleshooting
Pipeline failed
Jenkins console output lo first failed stage identify cheyyali.
Docker stage fail ayithe:
docker build -t mycompany/netflix:test .
Same build locally reproduce cheyyadaniki.
Registry stage fail ayithe:
docker login
Credentials/registry authentication verify.
Kubernetes stage fail ayithe:
kubectl get pods
Pods state check.
kubectl rollout status deployment/netflix
Rollout failure identify.
kubectl describe deployment netflix
Deployment events/configuration investigate.

PART 8 — ANSIBLE
Step 1 — Basic Commands
ansible --version
Ansible installed/version check.
ansible all -i inventory -m ping
Inventory lo unna servers Ansible connectivity test.
ansible all -i inventory -m command -a "uptime"
All servers lo uptime command execute chestundi.
ansible all -i inventory -m shell -a "df -h"
Remote servers disk usage check chestundi.
ansible-playbook -i inventory site.yml
Playbook execute chestundi.
ansible-playbook -i inventory site.yml --check
Changes actually apply cheyyakunda dry-run/check mode.
ansible-playbook -i inventory site.yml -v
More verbose execution information.

Step 2 — Ansible Real-time
ansible webservers -i inventory -m shell -a "systemctl status nginx"
Web servers lo Nginx status check.
ansible webservers -i inventory -m shell -a "df -h"
All web servers disk usage check.
ansible-playbook -i inventory deploy.yml
Application deployment playbook execute.

Step 3 — Ansible Troubleshooting
ansible all -i inventory -m ping
First connectivity test.
ansible-inventory -i inventory --graph
Inventory groups/hosts correct ga parse ayyaya check.
ansible-playbook -i inventory deploy.yml --syntax-check
Playbook syntax validate.
ansible-playbook -i inventory deploy.yml -vvv
Detailed debugging output.

PART 9 — AWS CLI
Step 1 — Basic Commands
aws --version
AWS CLI version check.
aws configure
AWS credentials/default region configuration setup.
aws sts get-caller-identity

Currently AWS CLI e account/user/role tho authenticated undo verify cheyyadaniki.
aws ec2 describe-instances
EC2 instances information choodadaniki.
aws s3 ls
S3 buckets list choodadaniki.
aws s3 ls s3://my-bucket
Specific bucket contents list.
aws s3 cp file.txt s3://my-bucket/
File S3 ki upload.
aws s3 cp s3://my-bucket/file.txt .
S3 nunchi local machine ki download.
aws ecr describe-repositories
ECR repositories information choodadaniki.
aws eks list-clusters
EKS Kubernetes clusters list choodadaniki.
aws eks update-kubeconfig --region ap-south-1 --name netflix-prod
EKS cluster credentials/context local kubectl configuration lo setup cheyyadaniki.

Step 2 — AWS Real-time
EKS cluster access
aws eks update-kubeconfig --region ap-south-1 --name netflix-prod
AWS EKS cluster ki kubectl access configure chestundi.
kubectl get nodes
EKS nodes accessible aa verify.
ECR image workflow
aws ecr get-login-password --region ap-south-1
ECR authentication token retrieve chestundi.
docker push <account>.dkr.ecr.ap-south-1.amazonaws.com/netflix:v1
Docker image ECR registry ki push chestundi after proper authentication/tagging.

Step 3 — AWS Troubleshooting
EKS access issue
aws sts get-caller-identity
Correct AWS identity use chestunnava check.
aws eks describe-cluster --name netflix-prod --region ap-south-1
Cluster status/configuration check.
kubectl get nodes
Kubernetes API access verify.
EC2 issue
aws ec2 describe-instances --instance-ids <instance-id>
Instance state/configuration inspect.

PART 10 — CI/CD + REAL-TIME MASTER FLOW
Real company lo nuvvu ekkuvaga ila commands combine chestav:
git pull origin main
Latest application code teesukuntav.
npm ci
Exact lock-file dependencies install chestav.
npm run build
Application build chestav.
docker build -t mycompany/netflix:v2.3.0 .
Production Docker image create chestav.
docker push mycompany/netflix:v2.3.0
Registry ki image upload chestav.
kubectl set image deployment/netflix netflix=mycompany/netflix:v2.3.0
Kubernetes Deployment ki new image cheptav.
kubectl rollout status deployment/netflix
New version successful ga rollout ayyinda check chestav.
kubectl get pods
Pods healthy aa check chestav.
kubectl logs <pod-name>
Application errors unnaya check chestav.
kubectl get svc
Service available aa check chestav.
kubectl get ingress
External routing available aa check chestav.
kubectl rollout undo deployment/netflix
New release fail ayithe previous version ki rollback chestav.

PART 11 — REAL-TIME TROUBLESHOOTING MASTER FLOW
Production lo **“Netflix down”** ani alert vachindani assume cheyyi.

First:
kubectl get pods -n netflix-prod
Pods Running, Pending, CrashLoopBackOff, ImagePullBackOff etc. lo unnaya identify chestav.
kubectl get deployment -n netflix-prod
Deployment desired replicas vs ready replicas compare chestav.
kubectl get svc -n netflix-prod
Service available aa check chestav.
kubectl get ingress -n netflix-prod
External routing resource check chestav.
kubectl get endpoints netflix-service -n netflix-prod
Service actually healthy Pods ni backend ga recognize chestunda check chestav.
kubectl describe pod <pod-name> -n netflix-prod
Pod events lo exact Kubernetes-level failure identify chestav.
kubectl logs <pod-name> -n netflix-prod
Application-level error identify chestav.
kubectl logs <pod-name> -n netflix-prod --previous
Previous crashed container logs check chestav.
kubectl describe deployment netflix -n netflix-prod
Deployment image, replicas, rollout/events check chestav.
kubectl rollout history deployment/netflix -n netflix-prod
Recent releases identify chestav.
If latest release problem ani confirm ayithe:
kubectl rollout undo deployment/netflix -n netflix-prod
Previous known-good version ki rollback chestav.
Tarvata:
kubectl rollout status deployment/netflix -n netflix-prod
Rollback successful aa verify chestav.
Finally:
kubectl get pods -n netflix-prod
All Pods healthy/Ready aa confirm chestav.

Core point: real-time DevOps lo commands separate ga memorize cheyyadam kanna, **issue → check → identify → fix → verify** flow important.
For example:
Application down
↓
kubectl get pods
↓
Pod problem?
↓
kubectl describe pod
↓
kubectl logs
↓
Root cause identify
↓
Fix configuration/image/resources
↓
kubectl apply / rollout
↓
kubectl rollout status
↓
Verify Pods + Service + Ingress
Idi actual job lo repeatedly use ayye pattern.